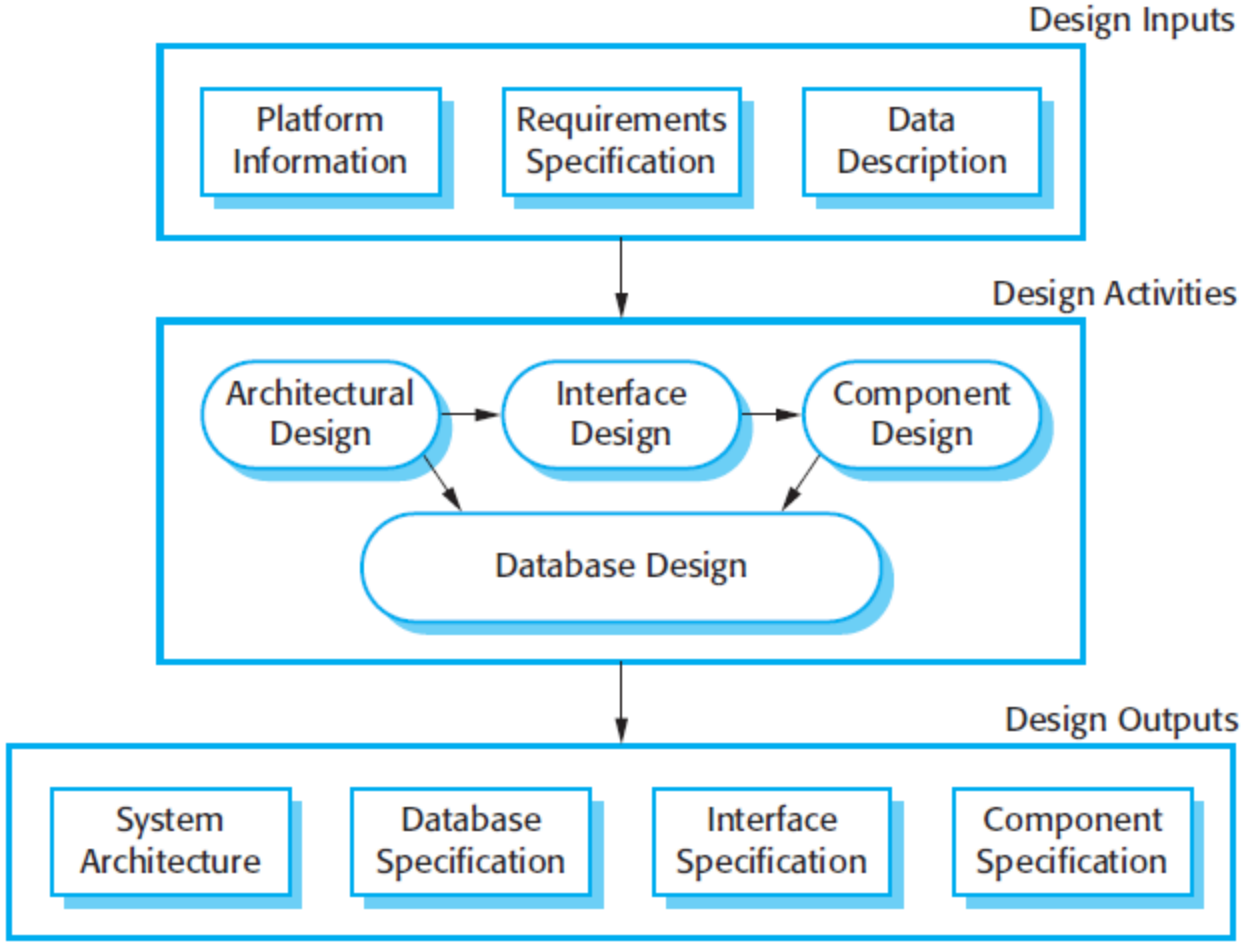
# Software Engineering

LECTURE 5

DR.DINA FAWZY

# A General Model of the Design Process



# Design Inputs

**Platform Information:** Most software interfaces with other software systems. These include the **operating system, database, middleware, and other application systems.** These make up the '**software platform', the environment** in which the software will execute. Information about this platform is an essential input to the design process, as designers must decide how best to **integrate it with the software's environment**.

The **requirements specification** is a description of the functionality the software must provide and its performance and dependability requirements.

**Data Description:** If the system is to process existing data, then the description of that data may be included in the platform specification; otherwise, the data description must be an input to the design process so that the system data organization to be defined.

# Design Process Activities

The activities in the design process **vary, depending on the type of system** being developed.

1. **Architectural design***,* where you identify the **overall structure** of the system, the **principal components** (sometimes called sub-systems or modules), their **relationships**, and how they are **distributed**.

2. **Interface design***,* where you define the **interfaces between system components**. This interface specification must be unambiguous. With a precise interface, a component can be used without other components having to know how it is implemented. Once interface specifications are agreed, the components can be designed and developed concurrently.

# Design Process Activities

3. **Component design***,* where you take each system component and design **how it will operate**. This may be a simple statement of the expected functionality to be implemented, with the specific design left to the programmer. Alternatively, it may be a list of changes to be made to a reusable component or a detailed design model. The design model may be used to automatically generate an implementation.

4. **Database design**, where you design **the system data structures** and how these are to be **represented in a database**. Again, the work here depends on whether an existing database is to be reused or a new database is to be created.

# Design Outputs

The detail and representation of these vary considerably.

For critical systems, detailed design documents setting out precise and accurate descriptions of the system must be produced.

If a model-driven approach is used, these outputs may mostly be diagrams.

Where agile methods of development are used, the outputs of the design process may not be separate specification documents but may be represented in the code of the program.

# Goal of Design Process

The design must implement **all of the explicit requirements** contained in the analysis model, and it must accommodate **all of the implicit requirements**.

The design must be a **readable**, **understandable** guide for those **who generate code** and for those **who test** and subsequently support the software.

The design should provide **a complete picture** of the software, addressing the **data, functional, and behavioral** domains from an implementation perspective.

# System Modelling

System modelling is the process of developing **abstract models** of a system, with **each model presenting a different view or perspective of that system.**

System modelling has generally come to mean representing the system using some kind of **graphical notation**, which is now almost always based on notations in the Unified modelling Language (UML).

# System Perspectives

You may develop different models to represent the system from **different perspectives.**

For example:

1. **An external perspective**, where you model the **context or environment** of the system.
2. **An interaction perspective** where you model the interactions between a system and its environment or between the components of a system.
3. **A structural perspective**, where you model the organization of a system or the structure of the data that is processed by the system.
4. **A behavioural perspective**, where you model the dynamic behaviour of the system and how it responds to events.

# Benefits

Help project teams **communicate**

- Communication between users, developers, analysts, testers, managers, and anyone else involved with a project

**Validate** the architectural design of the software

Be **independent** of particular programming languages and development processes

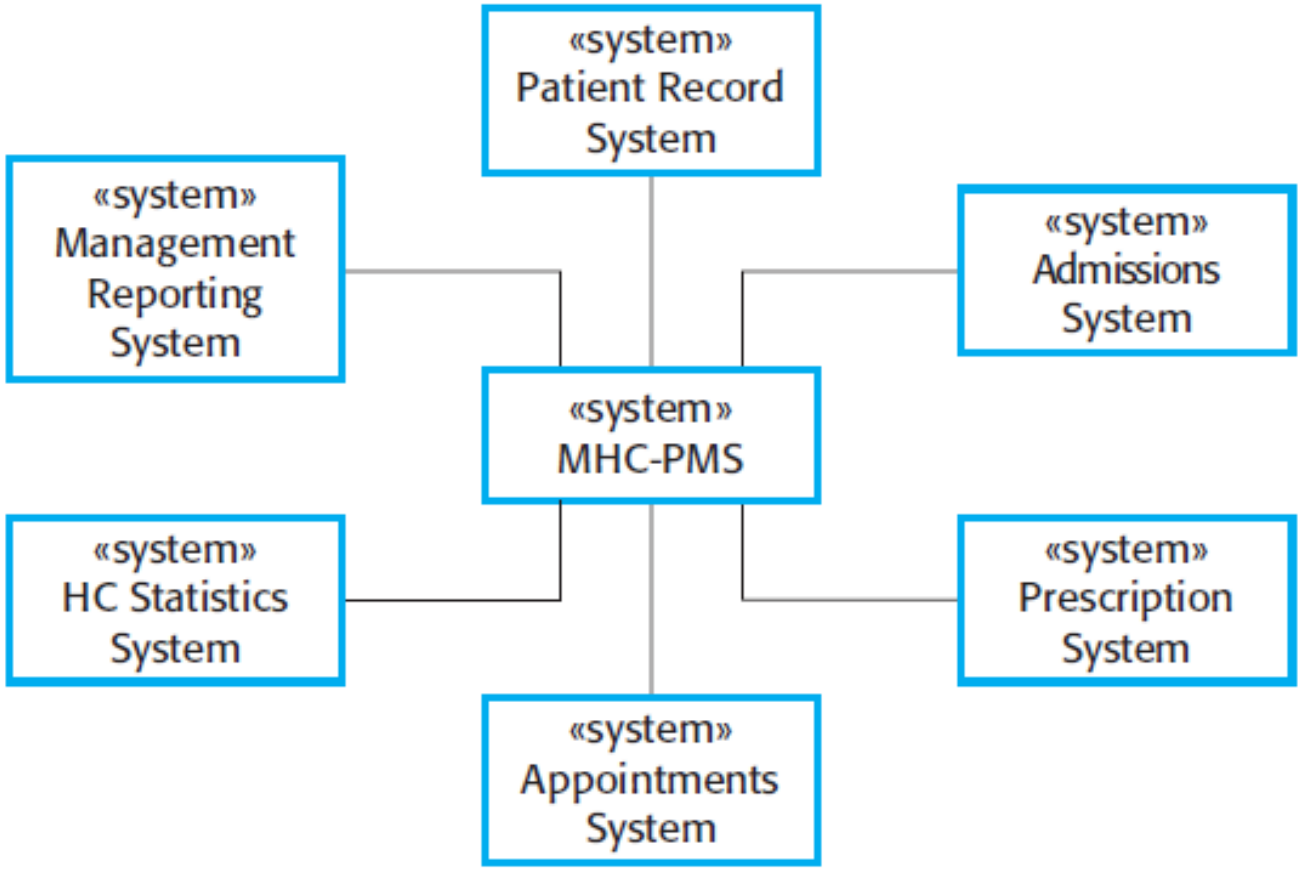
# Context Model

**Context models** normally show that **the environment** includes several other **automated systems**. However, **they do not show the types of relationships** between the systems in the environment and the system that is being specified.

- External systems might produce data for or consume data from the system.
- They might share data with the system, or they might be connected directly, through a network or not connected at all.
- They might be physically co-located or located in separate buildings.

Therefore, simple context models are used along with other models, such as **business process models.** These describe **human and automated processes in which particular software systems are used.**

# The Context of the MHC-PMS



# UML

UML stands for **Unified Modeling Language.** The Unified modelling Language is a set of more than 9 different diagram types that may be used to model software systems. It emerged from work in the 1990s on object-oriented modelling where similar object-oriented notations were integrated to create the UML. Variants have been proposed for more general system modelling.

- Use Case diagram
- Activity diagram
- Class diagram
- Sequence diagram
- Collaboration diagram
- Statechart diagram
- Deployment diagram

These model diagrams illustrate **different aspects of the system**

# Use Case modelling

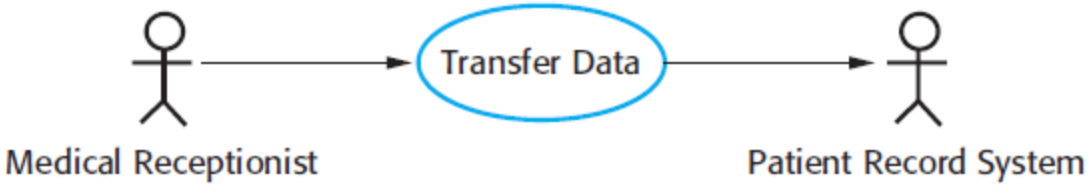
Use case modelling was originally developed by Jacobson et al. (1993) in the 1990s and was incorporated into the first release of the UML (Rumbaugh et al., 1999).

Use case modelling is widely used to support **requirements elicitation** because it describes what **a user expects from a system.**

# Use Case modelling

Each use case represents a discrete task (functionality) that involves external interaction with a system.

A **use case** is shown as an **ellipse** with the **actors** involved in the use case represented as **stick figures (user, system, time, hardware)**.



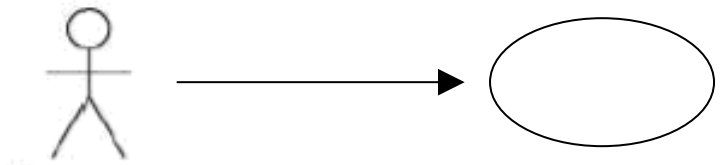
# Use Case Diagram

Use cases represent **system functionality**, the requirements of the system from **the user's perspective**

Use cases just focus on **automated processes**

Use case diagrams show the interactions between

- **Use cases:** what  the system should do
- **Actors:** anyone or anything **outside the system's scope but interacts with it** with the system (Individual, group, company,…)



# Actors

There are three types of actors:

- **Users** of the system (Physical person, or a user who will be directly using the system)
- **Other systems** that will interact with the system being built
- **Time (**Time becomes an actor when the passing of a certain amount of time triggers some event in the system (out of control))

# Use Case

The **functionality** the system will provide a value to the end user

They should be named in user terms, **not technical terms,** and should be meaningful to the customer

Use cases are an **implementation-independent**

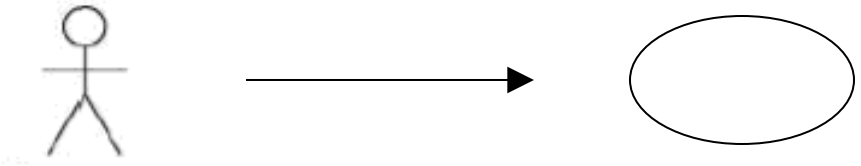
- High-level view of what the user expects from the system
- Focus on what the system should do, not how the system will

- Use Case diagrams aren't intended to show in which **order** the use cases are executed

# Association Relationship

Association relationship is used to show the relationship between a **use case and an actor only**

Every use case **must be initiated** by an actor, With the **exception** of use cases in includes and extends relationships

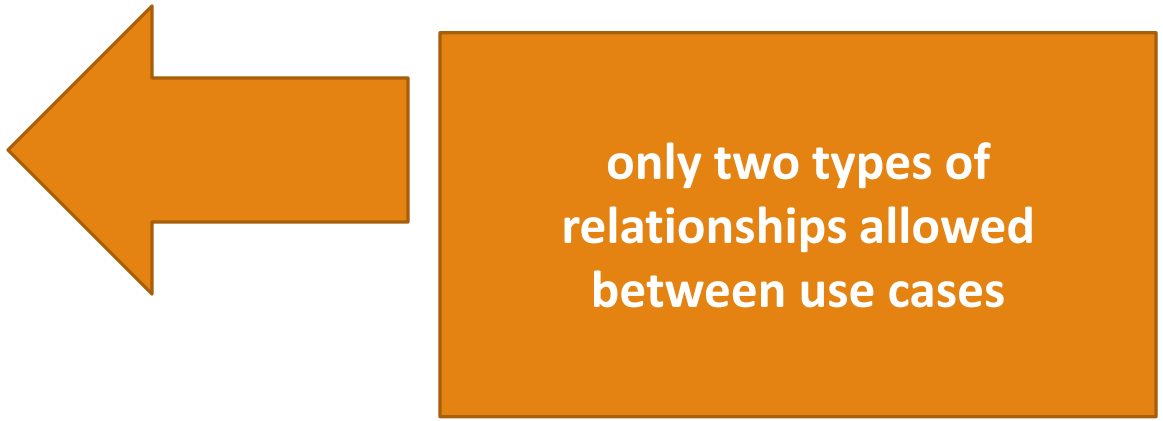


# Relationships

Association

Includes

Extends




Generalization

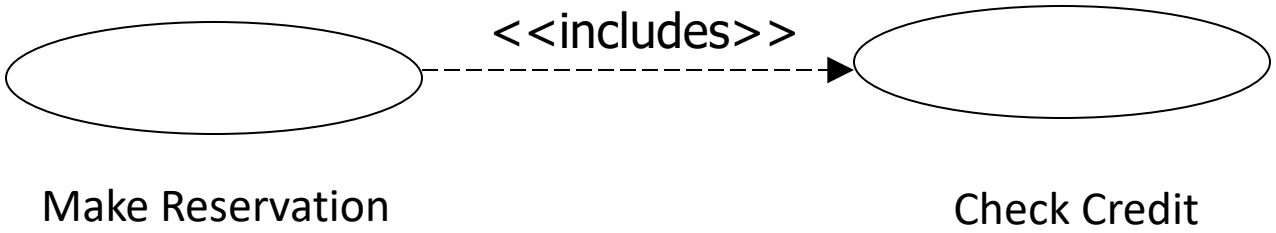
# Include Relationship

Includes relationship allows **one use case to use the functionality provided by another use case**

This relationship can be used in one of two cases:

- First, if two or more use cases have a **large piece of functionality that is identical**
- The second case where an includes relationship is helpful is a situation in which a **single use case has an unusually large amount of functionality**

An includes relationship suggests that one use case **always** uses the functionality provided by another

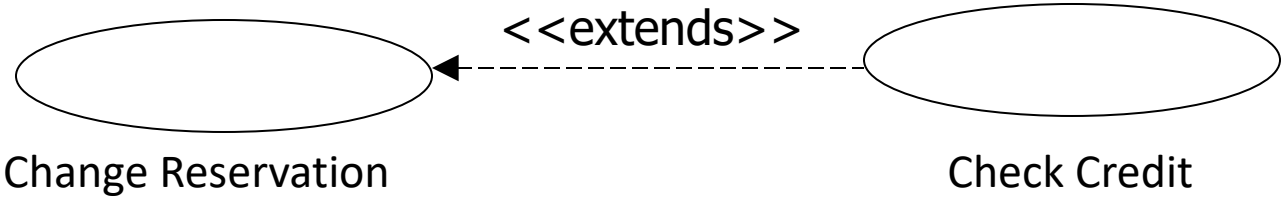



While the “Make Reservation" use case is running, "Check Credit" must run

# Extends Relationship

Extends relationship allows one use case **the option** to extend the functionality provided by another use case

It is very similar to an includes relationship, because in both of these types of relationships, you **separate some common functionality into its own use case**




While the "Change Reservation" use case is running, "Check Credit" runs if and only if the amount of the reservation has changed. If the amount has not changed, "Check Credit" does not need to run

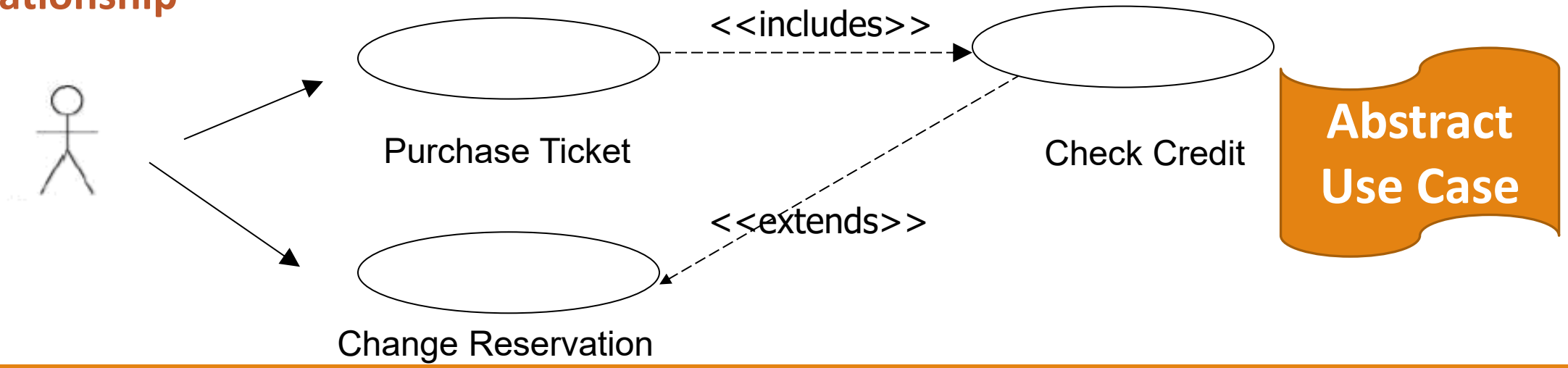
**IMP: The arrow is drawn from the use case that is optionally run ("Check Credit") to the use case that is being extended ("Change Reservation")**

# Abstract Use Case

An abstract use case is one that is **not started directly by an actor**

Instead, an abstract use case provides **some additional functionality that can be used by other use cases**

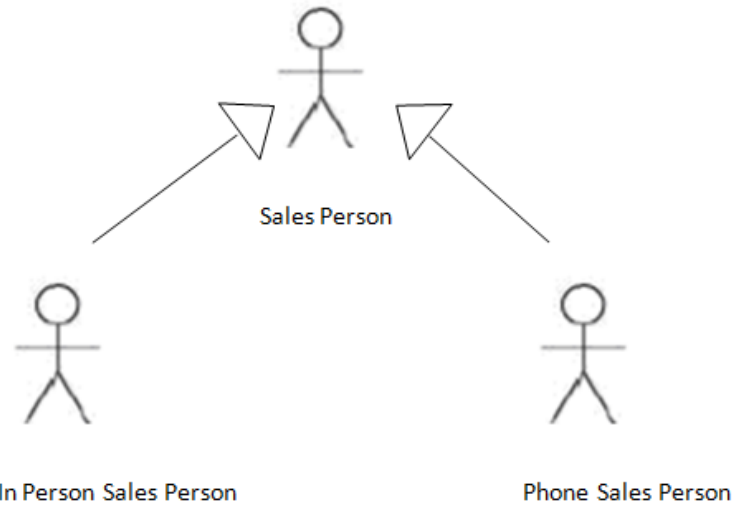
Abstract use cases are the use cases that **participate in an includes or extends relationship**

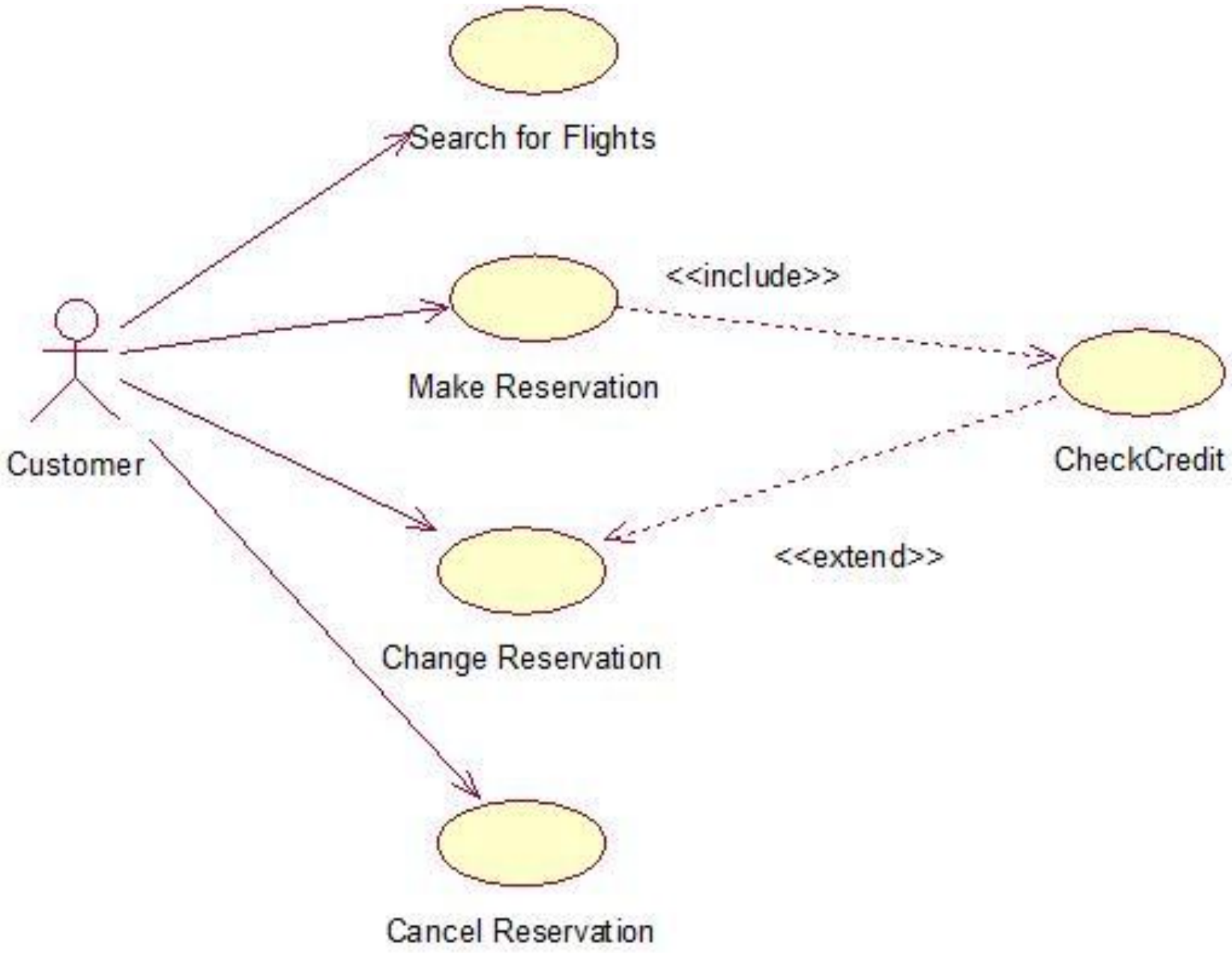


# Relationships: *Generalization*

Generalization relationship is used to show that several actors

For example, you may have two types of customers. If the type A customers will be initiating some use cases that type B customers will not, it's probably worth including the actor generalizations. If both types of customers use the same use cases, it's probably not necessary to show an actor generalization




Search for Flights
<<include>>
Make Reservation
Customer
CheckCredit
<<extend>>
Change Reservation
Cancel Reservation

# Thanks for Listening

ANY QUESTIONS!